LEMBAR KERJA PRAKTIKUM

Natural Language Processing

Preprocessing Teks Bahasa Indonesia

TUGAS INDIVIDU

Mata Kuliah	Natural Language Processing
Topik	Preprocessing Teks Bahasa Indonesia
Sifat Tugas	Individu — dikerjakan secara mandiri
Dikerjakan	Take-home setelah sesi praktikum
Dikumpulkan	Satu minggu setelah sesi praktikum
Bobot Nilai	60% dari nilai praktikum
DATA MAHASISWA	
Nama Lengkap	Abdul kharis
NIM	2388010004
Kelas	INF 6A

Petunjuk Penting

- ✦ Bacalah kode referensi pada Bagian A dengan seksama sebelum menjawab pertanyaan
- ✦ Kerjakan secara mandiri — jawaban harus mencerminkan pemahaman pribadi Anda
- ✦ Gunakan kalimat Anda sendiri, bukan hasil copy-paste dari teman atau internet
- ✦ Jawaban singkat tapi tepat lebih baik dari jawaban panjang tapi tidak relevan

BAGIAN A — Kode Referensi Pipeline Preprocessing

Perhatikan kode pipeline preprocessing lengkap berikut. Kode ini akan menjadi dasar untuk seluruh pertanyaan di lembar kerja ini. Baca dengan teliti setiap baris sebelum menjawab.

Keterangan Kode

Kode di bawah adalah pipeline preprocessing teks bahasa Indonesia dari awal hingga akhir. Terdiri dari 7 step: Load Data → Lowercasing → Remove Punctuation → Tokenisasi → Remove Stopwords → Stemming → Export.

Warna hijau = komentar penanda step | Warna gelap = kode Python aktif

```
# ========== SETUP AWAL
========== import pandas as pd
import nltk import re import
string
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory
```

```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
```

```
factory = StemmerFactory()
stemmer =
factory.create_stemmer()
stop_words_id = set(stopwords.words('indonesian'))
stop_words_id.update(['gw', 'ngga', 'nih', 'lah', 'sih', 'gan', 'klo'])
# ========== STEP 1: LOAD DATA ==========
data = {
    'id': [1, 2, 3],
    'ulasan': [
        'Produk ini sangat bagus banget! Saya sangat puas.',
        'Barangnya jelek, tidak sesuai deskripsi sama sekali!!!',
        'Gw suka banget sama produk ini nih! Recommended!'
    ]
}
df = pd.DataFrame(data)
```

```
# ========== STEP 2: LOWERCASING ==========
df['text_lower'] = df['ulasan'].str.lower()
```

```
# ========== STEP 3: REMOVE PUNCTUATION ==========
def remove_punctuation(text):
    text = re.sub(r'[!@#$%^&*()_+=\[\]{};:\'",.<>?/\\|-]', '',
    text) text = re.sub(r'\s+', ' ', text).strip() return text

df['text_clean'] = df['text_lower'].apply(remove_punctuation)
```

```
# ========== STEP 4: TOKENIZATION ==========
df['tokens'] = df['text_clean'].apply(word_tokenize)
```

```
# ========== STEP 5: REMOVE STOPWORDS ==========
def remove_stopwords(tokens, stop_words):
    return [word for word in tokens if word.lower() not in stop_words]
df['tokens_no_stop'] = df['tokens'].apply( lambda
    x: remove_stopwords(x, stop_words_id))
```

```
# ========== STEP 6: STEMMING ==========
def apply_stemming(tokens, stemmer):
    return [stemmer.stem(word) for word in tokens]
df['tokens_stemmed'] = df['tokens_no_stop'].apply(
    lambda x: apply_stemming(x, stemmer))
```

```
# ========== STEP 7: HASIL FINAL ==========
df['text_final'] = df['tokens_stemmed'].apply(lambda x: ' '.join(x))
```

```
# ========== EXPORT ==========
df[['id','ulasan','text_final']].to_csv('ulasan_preprocessed.csv', index=False)
```

BAGIAN B — Pemahaman Pipeline

Berdasarkan kode referensi di Bagian A, jawab pertanyaan berikut. Anda boleh merujuk kembali ke kode kapan saja.

B.1 Fungsi Setiap Step

Lengkapi tabel berikut berdasarkan pemahaman Anda terhadap kode referensi:

No	Nama Step	Fungsi Python yang Digunakan	Tujuan Step (jelaskan dengan kalimat sendiri)
1	Load Data	pd.DataFrame()	Data ulasan dimasukkan ke DataFrame supaya bisa diproses step-by-step.
2	Lowercasing	.str.lower()	Menyamakan bentuk kata agar tidak ada perbedaan makna hanya karena huruf besar-kecil.
3	Remove Punctuation	re.sub()	Membersihkan tanda baca yang tidak dipakai dalam analisis teks.
4	Tokenisasi	word_tokenize()	Memecah teks menjadi unit kata sesuai kebutuhan preprocessing.
5	Remove Stopwords	list comprehension	Menghilangkan kata umum yang tidak memberi informasi penting.
6	Stemming	stemmer.stem()	Mengembalikan kata ke bentuk dasar agar variasi kata tidak dianggap berbeda.
7	Export	.to_csv()	Menyimpan hasil akhir preprocessing untuk tahap lanjutan.

B.2 Urutan dan Ketergantungan Antar Step

Mengapa Step 3 (Remove Punctuation) harus dilakukan SEBELUM Step 4 (Tokenisasi)? Apa yang terjadi jika urutannya dibalik? Berikan contoh konkret:

Karena teks dibersihkan dulu baru dipecah. Agar menghasilkan kata. Bukan kata + sampah
EX: "buruk!!!" Remove punctuation jadi "buruk" token: ['buruk']
Jika dibalik tanda baca terbawa kedalam token menjadi ['buruk!!!']

Mengapa Step 2 (Lowercasing) diletakkan di awal sebelum step lainnya? Apakah boleh dilakukan setelah tokenisasi? Jelaskan:

Untuk konsistensi. Semuanya I huruf kecilkan dulu baru proses. Bisa setelahnya tapi kurang optimal karena harus diulang tiap token

B.3 Input dan Output Setiap Step

Perhatikan ulasan berikut: "Tidak Recommended!!! Kualitas buruk dan penggiriman lambat!!!"

Isi tabel berikut dengan perkiraan hasil transformasi di setiap step (Anda TIDAK perlu menjalankan kode — jawab berdasarkan pemahaman):

No	Step	Hasil Transformasi (perkiraan)
1	Teks Asli	"Tidak Recommended!!! Kualitas buruk dan penggiriman lambat!!!"
2	Lowercasing	"tidak recommended!!! kualitas buruk dan penggiriman lambat!!!"
3	Remove Punctuation	"tidak recommended kualitas buruk dan penggiriman lambat"
4	Tokenisasi	['tidak','recommended','kualitas','buruk','dan','penggiriman','lambat']
5	Remove Stopwords	['recommended','kualitas','buruk','penggiriman','lambat']
6	Stemming	['recommend','kualitas','buruk','kirim','lambat']
7	Text Final	"recommend kualitas buruk kirim lambat"

Dari tabel di atas, informasi apa yang hilang setelah melalui seluruh pipeline? Apakah hilangnya informasi tersebut bermasalah atau tidak? Jelaskan:

Tanda seru, jiwa atau ekspresi, “tidak”(stopwords) bentuk kata pengirim jd kirim
Kasus ini pengaruh karena ada hilangya tidak menjadikan pnegative jadi positive. Tapi tegantung kebutuhan apakah normalisasi atau sentiment

BAGIAN C — Kemampuan Memahami Kode Pipeline

Petunjuk Bagian C

Bagian ini terdiri dari 3 level soal kode dengan tingkat kesulitan meningkat.
C.1 Fill-in-the-blank — isi bagian kode yang dikosongkan (Level: Dasar)
C.2 Trace kode — telusuri output tanpa menjalankan kode (Level: Menengah)
C.3 Perbaiki kode — temukan dan jelaskan bug dalam kode (Level: Lanjut)
Anda TIDAK diperbolehkan menjalankan kode untuk menjawab bagian ini.

C.1 Fill-in-the-Blank — Lengkapi Implementasi Kode

Implementasikan pipeline preprocessing untuk teks tweet berbahasa Indonesia. Tweet berbeda dengan ulasan produk karena mengandung elemen khusus seperti hashtag, mention, URL, dan emoji yang perlu ditangani sebelum pipeline standar dijalankan.

Petunjuk

Lengkapi setiap fungsi yang ditandai # TODO dengan kode Python yang tepat.
Gunakan library re (regex) untuk menangani pola teks khusus.

Setelah semua fungsi selesai, jalankan pipeline lengkap pada contoh tweet di bawah. Anda BOLEH merujuk kode referensi di Bagian A sebagai panduan pipeline standar.

```
# Contoh tweet yang akan diproses: tweet = '@tokopedia Produknya bagus tapi
ongkirnya mahal bgt  #kecewa #onlineshop https://t.co/xxx'
```

```
# LANGKAH 1 — Hapus URL
def hapus_url(text):
    # TODO: Hapus semua URL yang diawali http/https
    # Hint: gunakan re.sub() dengan pola
    r'https?\S+' return text

# LANGKAH 2 — Hapus Mention (@username)
def hapus_mention(text):
    # TODO: Hapus semua kata yang diawali tanda @
    # Hint: pola mention adalah @diikuti_karakter_tanpa_spasi
    return text

# LANGKAH 3 — Ubah Hashtag menjadi Kata Biasa
def proses_hashtag(text):
    # TODO: Ubah #topik menjadi 'topik' (hapus tanda # saja, kata tetap ada)
    # Hint: berbeda dengan hapus_mention — kata hashtag tetap dipertahankan
    return text

# LANGKAH 4 — Hapus Emoji
def hapus_emoji(text):
    # TODO: Hapus semua karakter emoji dari teks
    # Hint: emoji berada di rentang unicode, gunakan re.sub() dengan pola:
    # r'[\U0001F600-\U0001F64F\U0001F300-\U0001F5FF\U0001F680-
    \U0001F6FF]+' return text

# LANGKAH 5 — Pipeline Standar (dari kode referensi Bagian A)
def pipeline_standar(text):
    # TODO: Terapkan 5 step pipeline standar secara berurutan:
    # (1) lowercase, (2) remove punctuation, (3) tokenisasi, # (4) hapus
    stopwords, (5) stemming, lalu gabungkan kembali return text

# PIPELINE LENGKAP — Gabungkan semua langkah
def preprocess_tweet(text):
    # TODO: Panggil semua fungsi di atas secara berurutan
    # Langkah 1 → 2 → 3 → 4 → 5

    return text
# Jalankan pipeline pada contoh tweet
hasil = preprocess_tweet(tweet)
print('INPUT :', tweet)
print('OUTPUT:', hasil)
```

Tulis implementasi lengkap semua fungsi di atas (hapus_url, hapus_mention, proses_hashtag, hapus_emoji, pipeline_standar, preprocess_tweet):

```
import re
from nltk.tokenize import word_tokenize
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory

factory = StemmerFactory()
stemmer = factory.create_stemmer()
```

```
stopwords = set(['dan', 'tapi', 'yang', 'di', 'ke', 'dari', 'itu', 'ini', 'nya'])

# LANGKAH 1 — Hapus URL
def hapus_url(text):
    text = re.sub(r'https?\S+', '', text)
    return text

# LANGKAH 2 — Hapus Mention (@username)
def hapus_mention(text):
    text = re.sub(r'@\S+', '', text)
    return text

# LANGKAH 3 — Ubah Hashtag menjadi Kata Biasa
def proses_hashtag(text):
    text = re.sub(r'#', '', text)
    return text

# LANGKAH 4 — Hapus Emoji
def hapus_emoji(text):
    text = re.sub(
        r'[\U0001F600-\U0001F64F'
        r'\U0001F300-\U0001F5FF'
        r'\U0001F680-\U0001F6FF]+',
        '',
        text
    )
    return text

# LANGKAH 5 — Pipeline Standar
def pipeline_standar(text):
    text = text.lower()
    text = re.sub(r'[^\w\s]', '', text)
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if t not in stopwords]
    tokens = [stemmer.stem(t) for t in tokens]
    text = ' '.join(tokens)
    return text

# PIPELINE LENGKAP
def preprocess_tweet(text):
    text = hapus_url(text)
    text = hapus_mention(text)
    text = proses_hashtag(text)
    text = hapus_emoji(text)
    text = pipeline_standar(text)
    return text
```

Tuliskan output yang dihasilkan oleh preprocess_tweet() untuk contoh tweet di atas. Jelaskan step by step transformasi yang terjadi:

Input @tokopedia Produknya bagus tapi ongkirnya mahal bgt (emot) , #kecewa # onlineshop https://t.co/xxx
hapus_url → @tokopedia Produknya bagus tapi ongkirnya mahal bgt (emot), #kecewa #onlineshop
hapus_mention→ Produknya bagus tapi ongkirnya mahal bgt (emot), #kecewa #onlineshop
proses_hashtag→ Produknya bagus tapi ongkirnya mahal bgt (emot), kecewa onlineshop
hapus_emoji → Produknya bagus tapi ongkirnya mahal bgt kecewa onlineshop
lowercase → produknya bagus tapi ongkirnya mahal bgt kecewa onlineshop

hapus tanda baca → produknya bagus tapi ongkirnya mahal bgt kecewa onlineshop
tokenisasi → ['produknya','bagus','tapi','ongkirnya','mahal','bgt','kecewa','onlineshop']
hapus stopwords → ['produknya','bagus','ongkir','mahal','bgt','kecewa','onlineshop']
stemming → ['produk','bagus','ongkir','mahal','bgt','kecewa','onlineshop']
output akhir → produk bagus ongkir mahal bgt kecewa onlineshop

Mengapa langkah 1–4 (hapus URL, mention, hashtag, emoji) harus dilakukan SEBELUM pipeline standar? Apa yang terjadi jika urutannya dibalik?

Karena teks di cleaning baru proses agar pipeline standar itu di teks normal bukan yang mentah
Jika elemen khusus tweet blm dihapus :URL jadi token, mention (@username) sebagai kata, emoji jadi token aneh atau error, hashtag hilang maknanya atau terpotong

---

BAGIAN D — Analisis Kritis terhadap Pipeline

D.1 Analisis Kode Referensi

Pada setup awal, kode mengunduh tiga resource NLTK: 'punkt', 'stopwords', dan 'wordnet'. Namun dalam pipeline ini tidak ada tahap lemmatization yang membutuhkan 'wordnet'. Mengapa menurut Anda 'wordnet' tetap diunduh? Apakah ini masalah?

Maybe template atau hasil dari Ai, atau source lama
Dan gak ada masalah cuman jadi resource percuma tidak clean efisien

Perhatikan baris ini pada kode: stop_words_id.update(['gw', 'ngga', 'nih', 'lah', 'sih', 'gan', 'klo']). Mengapa kata-kata ini tidak ada dalam stopwords bawaan NLTK untuk bahasa Indonesia? Jelaskan:

Nltk itu Bahasa formal sesuai kbbi mungkin
Jadi kalimat informal slang juga kaliat chat tidak termasuk jadi harus dibuat stopwords agar relevan

Fungsi remove_punctuation menggunakan regex: re.sub(r'[!@#$%^&*()_+=...', '', text). Menurut Anda, apa kelemahan pendekatan regex seperti ini dibanding menggunakan re.sub(r'[^\w\s]', '', text)? Kata atau Karakter apa yang mungkin lolos?

Manual symbol satu persatu rawan keliru
Akibatnya emoji, symbol Unicode bisa lolos tokenisasi
Pendekatan ke 2 lebih aman karena non huruf dan non spasi dihapus

D.2 Analisis Stopwords

Perhatikan dua kalimat berikut yang diproses dengan kode referensi:

```
Kalimat A: "Produk ini bagus"
```

```
Kalimat B: "Produk ini tidak bagus"
```

Jika kata 'tidak' ada dalam stopwords dan dihapus, apa yang terjadi pada makna kedua kalimat tersebut setelah diproses? Tuliskan hasil akhir kedua kalimat dan jelaskan implikasinya untuk analisis sentimen:

A "Produk ini bagus" → produk bagus
B "Produk ini tidak bagus" → produk bagus
Makna sama meski sentiment berbeda
Dampak negative jadi positive, jika model untuk sentiment bisa salah klasifikasi

Kapan menurut Anda kata 'tidak' BOLEH dihapus sebagai stopwords dan kapan TIDAK BOLEH? Berikan skenario penggunaan NLP yang berbeda untuk masing-masing jawaban:

BOLEH : topic modelling Clustering, pencarian kata kunci Fokusnya ke topik, bukan emosi.
TIDAK BOLEH : analisis sentiment, opinion mining,review scoring Karena tidak adalah kata negasi yang menentukan makna.

D.3 Analisis Stemming

Berikut adalah beberapa kata dari dataset ulasan produk yang diproses oleh Sastrawi:

No	Kata Asli	Hasil Stemming Sastrawi	Tepat? (✓/✗) + Penjelasan
1	kualitasnya	kualitas	✓ – -nya dibuang, makna tetap.
2	penggiriman	kirim	✓ – pe-an dibuang, kata dasar masih nyambung.
3	recommended	recommend	✗ – kata Inggris, Sastrawi untuk indo.
4	terjangkau	jangkau	✓ – ter- dibuang, arti inti masih sama.
5	pembelian	beli	✓ – pe-an dibuang, makna tetap beli.
6	kecewa	kecewa	✓ – kata dasar

Berdasarkan tabel di atas, jelaskan apa yang dimaksud dengan 'overstemming' dan 'understemming'. Berikan contoh dari tabel tersebut jika ada:

Overstemming .stemming over/ berlebihan, kata yang seharusnya punya makna beda jadi satu bentuk dasar dan maknanya kepotong. penggiriman → kirim teknis sama tapi makna beda

Understemming. stemming kurang, kata yang seharusnya bisa disatukan tetap dibiarkan beda. kecewa → kecewa Kata ini tidak berubah

D.4 Studi Kasus: Analisis Sentimen

Bayangkan Anda bekerja di tim data science sebuah marketplace dan diminta membangun model klasifikasi ulasan (positif/negatif). Dataset berisi 50.000 ulasan dalam bahasa Indonesia informal.

Berdasarkan kode referensi, urutkan tahap preprocessing dari yang paling kritis hingga paling bisa dilewati untuk kasus ini. Berikan justifikasi untuk setiap urutan:

Lowercasing Wajib. Tanpa ini, kata yang sama bisa kebaca beda.
Tokenisasi Kalau berantakan, step lain ikut rusak. Fondasi proses.
Stopwords (selektif) Penting, tapi hati-hati. Kata tidak nggak boleh dibuang kalau sentimen.
Normalisasi teks informal untuk dataset informal (krusial). Contoh: bgt → banget, gk → tidak.
Remove punctuation Membersihkan noise, tapi dampaknya ke sentimen.
Stemming Berguna, tapi bisa dikompromi., terlalu agresif bisa nurunin akurasi.

Adakah tahap dalam kode referensi yang menurut Anda perlu DIMODIFIKASI atau DITAMBAHKAN untuk dataset ulasan informal berskala besar? Jelaskan apa yang perlu diubah dan mengapa:

Stopwords custom. Stopwords bawaan nggak cukup buat bahasa informal.
Emoji handling, Jangan cuma dihapus, bisa dimapping ke positif/negatif.
Normalisasi slang & typo. data informal skala besar pasti penuh variasi.
Handling negation Contoh: tidak bagus jangan dipisah. Bisa bigram: tidak_bagus.

BAGIAN D — Analisis Statistik Hasil Preprocessing

Berdasarkan hasil praktikum kelompok Anda, gunakan data statistik yang sudah dikumpulkan untuk menjawab pertanyaan analisis berikut.

D.1 Interpretasi Data

Salin data statistik dari lembar kerja kelompok Anda ke tabel berikut:

ID	Penggalan Teks Asli	Token Asli	Token Final	% Reduksi
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
	RATA-RATA			

Ulasan mana yang memiliki persentase reduksi token terbesar dan terkecil? Analisis mengapa ulasan tersebut mengalami reduksi paling banyak — kaitkan dengan jenis kata yang mendominasi ulasan tersebut:

Pilih dua ulasan yang menurut Anda paling berbeda kualitas hasilnya setelah preprocessing (satu yang hasilnya baik, satu yang hasilnya kurang baik). Jelaskan alasan Anda dan apa yang membuat perbedaan tersebut terjadi:

SOAL BONUS— Opsional (+10 Poin Bonus)

Petunjuk

Bagian ini bersifat OPSIONAL. Dikerjakan secara mandiri di rumah.
Nilai tambahan maksimal: +10 poin bonus di atas nilai individu.
Jawaban harus orisinal — bukan hasil AI atau copy dari sumber lain.

Pengayaan 1 — Telaah Pipeline dari Sudut Pandang Berbeda

Kode referensi dirancang untuk ulasan produk e-commerce. Bayangkan pipeline yang sama diterapkan pada dua jenis teks berbeda:

Aspek	Teks Berita Formal	Tweet Informal
Lowercasing — masih relevan?	Ya, untuk konistensi	Ya, variasi huruf di emsos lebih liar
Stopwords — perlu modifikasi?	Sedikit, teks formal dengan stopwords formal cukup	Ya, karena banyak slang dll
Stemming — tetap cocok?	Ya, secara umum	Tergantung data, dan focus, makna bisa bberubah
Step tambahan yang diperlukan	Cukup	Normalisasi, slang, emoji, negasi dll

Dari tabel di atas, simpulkan: apakah satu pipeline bisa digunakan untuk semua jenis teks, atau setiap domain perlu pipeline yang berbeda? Jelaskan:

1 pipeline tidak untuk semua teks
Jika formal Bahasa rapi, struktur jelas prcesing standar cukup
Pipeline informal Bahasa campurm slag, emoji negasi, emosi perlu processing tambahan
Tiap domain perlu pipeine berbeda

Pengayaan 2 — Mini Research

Cari satu paper atau artikel ilmiah (tahun 2020–2025) tentang preprocessing NLP untuk bahasa Indonesia. Tuliskan judul, penulis, tahun, dan bandingkan metode preprocessing yang digunakan dengan kode referensi di Bagian A — apa yang sama dan apa yang berbeda?

Laporan Analisis Artikel Ilmiah dan Perbandingan Preprocessing

1. Identitas Artikel Ilmiah

Judul: Analisa Sentimen Data Text Preprocessing pada Data Mining dengan Menggunakan Machine Learning

Link ***https://journal.ubm.ac.id/index.php/jbase/article/viewFile/3000/2151***.

Penulis: Bhustomy Hakim

Tahun: 2021

2. Perbandingan Metode Preprocessing

A. Persamaan

Lowercasing

Keduanya mengubah teks ke huruf kecil supaya konsisten.

Remove punctuation

Sama-sama membersihkan tanda baca biar nggak ganggu token.

Stopwords removal

Kata umum yang tidak punya makna penting dibuang.

Stemming

Sama-sama mengubah kata ke bentuk dasar untuk nyederhanakan fitur.

Intinya: alur dasar preprocessingnya mirip.

B. Perbedaan

Bahasa & Library

Artikel fokus ke Bahasa Inggris dan pakai NLTK + Porter Stemmer, sedangkan kode A pakai Bahasa Indonesia + Sastrawi.

Stopwords

Kode A nambah stopwords manual (gw, ngga, nih, dll) karena data informal, artikel hanya pakai stopwords standar NLTK.

Karakter Khusus

Artikel menghapus tag HTML (misalnya <br>) karena sumber data dari web (IMDB). Di kode A, step ini tidak ada.

Fokus Analisis

Artikel lebih fokus ke pengaruh preprocessing ke akurasi Naïve Bayes, sedangkan kode referensi fokus ke alur pipeline preprocessing itu sendiri.

3. Kesimpulan Temuan Artikel

Berdasarkan eksperimen di artikel, data baseline (tanpa preprocessing berat) justru menghasilkan akurasi tertinggi (0,83993).

Ini menunjukkan preprocessing yang terlalu agresif bisa menghilangkan konteks dan malah menurunkan performa model, artinya preprocessing itu harus proporsional, bukan makin banyak step makin bagus.

Kesimpulan Akhir

Pipeline di kode referensi:

lebih cocok untuk bahasa Indonesia informal, fleksibel karena stopwords bisa dikustom

Sedangkan artikel menunjukan:

preprocessing tidak boleh berlebihan dan harus disesuaikan sama data dan tujuan model. Karena Satu pipeline tidak bisa dipakai mentah untuk semua kasus.

RUBRIK PENILAIAN — INDIVIDU

Bgn	Komponen	Fokus Penilaian	Kriteria Jawaban Baik	Bobot	Nilai
B	Pemahaman Pipeline	Urutan, fungsi & ketergantungan step	Mampu menjelaskan tujuan setiap step dan alasan urutannya dengan tepat	20%	
C.1	Fill-in-the-Blank	Pengetahuan sintaks & fungsi Python/NLTK	Semua bagian kosong diisi dengan tepat tanpa menjalankan kode	15%	
C.2	Trace Kode	Kemampuan mengikuti alur logika kode	Nilai variabel di setiap step akurat dan analisis kehilangan informasi tepat	15%	
C.3	Perbaiki Kode	Pemahaman konsep & identifikasi kesalahan	Semua bug ditemukan dengan penjelasan dampak yang logis dan benar	15%	
D	Analisis Kritis	Kelebihan & kelemahan pipeline	Identifikasi masalah kode disertai justifikasi berbasis konsep yang kuat	15%	
E	Analisis Statistik	Interpretasi data hasil praktikum	Mampu membaca pola data dan menghubungkan dengan konsep preprocessing	10%	
F	Refleksi	Kedalaman pemahaman personal	Refleksi mendalam, ditulis dengan kalimat sendiri, menunjukkan pemahaman menyeluruh	10%	
	TOTAL			100%	
	Bobot Praktikum		Tugas individu = 60% dari nilai praktikum	60%	

Catatan / Feedback Dosen: